Task 4: Zero-Shot Entailment Transfer via Prompt Engineering

1. Task Description

In this task, my objective was to build a visual entailment transfer pipeline capable of generalizing unseen entailment relationships and concept types using prompt engineering. This approach was designed to avoid the expensive retraining of the full multimodal model.

The core requirements were:

- Frozen Backbone & Lightweight Head: Use a frozen ViT image encoder and a frozen BERT text encoder, training only a lightweight adapter-based classifier (~100k params) on top of their fused representations.
- Prompt Design: Design prompt templates to test unseen entailment relationships, experimenting with variations in specificity level, concept framing, and negation patterns.
- Evaluation: Test the transfer on 3-5 held-out datasets and custom synthetic pairs. Measure the zero-shot accuracy against a 1-shot adaptation.
- Deliverables: Create a prompt engineering guide documenting which framings work best for different concept types.
- Benefit: Enable entailment checking on new concepts without retraining, reducing the data collection burden for new domains by adapting model behavior through natural language prompts.

2. Procedure and Technical Implementation

2.1 Frozen Backbone and Feature Extraction

- I implemented a FrozenViTTextEncoder combining google/vit-base-patch16-224 as the visual encoder and bert-base-uncased as the text encoder.
- I froze all parameters in both base models by setting requires_grad = False to ensure the multimodal backbone only acted as a fixed feature extractor.
- I created a FeatureBuilder to extract the [CLS] token from both the image and the prompted text, concatenating and L2-normalizing them into a single 1536-dimensional fused feature representation.

2.2 The Lightweight Adapter-Classifier Head & Architectural Pivot

- Architectural Pivot (Efficiency vs. Expressiveness): I intentionally replaced the previous attention-based cross-modal fusion head with a lightweight adapter. While the original fusion head provided the explicit, fine-grained interaction needed for exact entailment reasoning, it was computationally heavy. By simply concatenating the ViT and BERT [CLS] embeddings instead, I made the pipeline significantly faster and cheaper to train, trading precise relational reasoning for broad multimodal compatibility.
- Feature Caching (Eliminating Redundant Computation): To massively accelerate the evaluation loop, I implemented an embedding cache within the FeatureBuilder. Because the model evaluates the exact same images across five different prompt templates, the frozen ViT processes each raw image only once. The resulting 768-dimensional visual [CLS] embedding is stored directly in memory (self.image_cache), completely eliminating the need to pass the image through the heavy ViT layers again and again for every text variation.
- Bottleneck Adapter: With the base encoders completely frozen, the trainable parameter budget was restricted to this new head. I built a BottleneckAdapter that projected the 1536-dimensional input down to a 32-dimensional bottleneck, applied GELU and dropout, and projected it back up with a learned residual scale.
- SwiGLU Classifier: To actively filter noisy prompt effects, I replaced standard linear classification with a SwiGLU_MLP classifier, utilizing a Gated Linear Unit (SiLU(x1) * x2) to model nonlinear interactions before outputting the 3-class logits.
- Training Execution: I trained this lightweight head using the AdamW optimizer, implementing early stopping, weight decay, and gradient clipping to ensure stability.
-

2.3 Concept Extraction and Lexicon Mining

- To test prompt efficacy across semantic categories, I built a build_dataset_concept_lexicon function that scanned the training hypotheses to mine frequent verbs, nouns, and adjectives using NLTK (or fallback suffix rules) alongside spatial and numeric cues.
- I implemented a hybrid analyze_concept function that combined rule-based regex matching with dependency parsing via spaCy.
- This hybrid analyzer successfully assigned every test hypothesis to one of five prioritized buckets: negation, count, spatial, action, or object_or_attribute.

2.4 Prompt Engineering Design

- I designed a PROMPT_TEMPLATES dictionary containing five distinct text formulations to wrap the extracted concept anchor and hypothesis text before passing it to BERT:
    - base: The baseline formulation ("Image shows {concept}. Statement: {text}...").

- low_specificity: Instructed the model to focus on broad categories and general scene semantics.
- high_specificity: Commanded the model to verify fine-grained attributes and exact sub-categories.
- concept_framing: Introduced a structural "Visual Anchor: [{concept}]" before the statement to guide the reasoning logic.
- negation_explicit: Explicitly instructed the model to carefully handle statements containing polarity and negation cues.

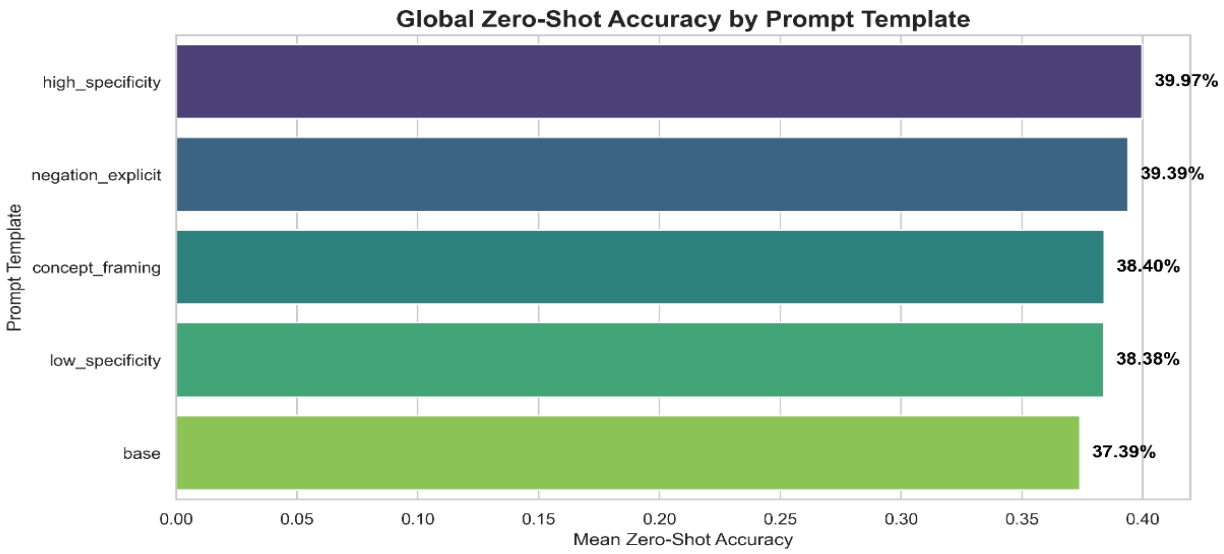
2.5 Synthetic Datasets and Evaluation Suite

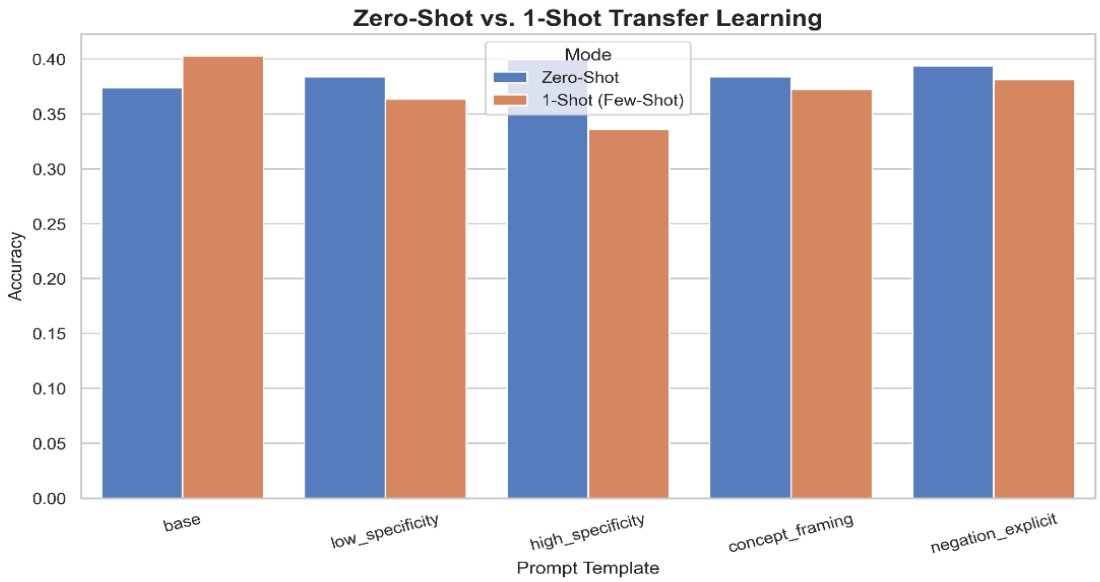
- To rigorously test the transfer, I used build_synthetic_sets to generate custom synthetic stress sets from the test split, including an Identity Entailment set (where the hypothesis equals the premise), a Negation Contradiction set, and a Mismatch Neutral set (where hypotheses were shuffled).
- I executed a strict zero-shot evaluation protocol (evaluate_head) by swapping prompt templates without applying any additional retraining to the adapter.
- Finally, I implemented a one-shot evaluation protocol (run_one_shot_eval) that copied the frozen adapter, fine-tuned it on exactly three support examples (one per entailment class), and tested it against the query sets to measure the impact of minimal supervised adaptation.

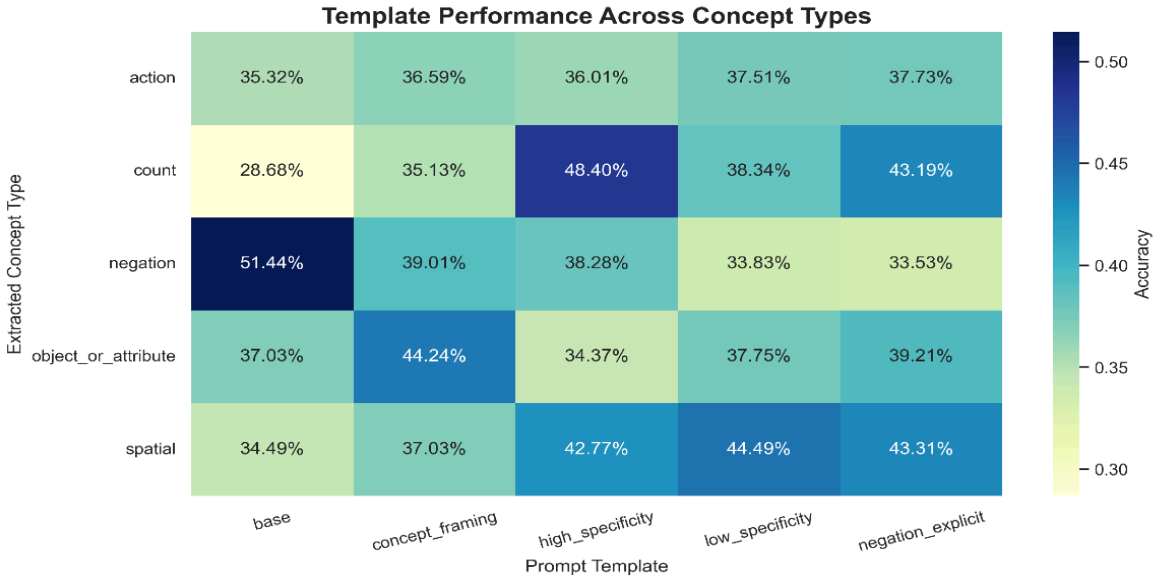
3.Results and Evidence

```
==========================================================================
🎯 FINAL PROMPT ENGINEERING GUIDE MATRIX 🎯
==========================================================================
Prompt Variation  Action Acc %  Spatial Acc %  Attribute Acc %  Object Acc %  Overall Acc %
        BASELINE         46.81          50.47            45.11         49.35          48.37
     SPECIFICITY         48.81          46.15            47.74         49.43          48.93
 CONCEPT_FRAMING         47.12          45.58            48.87         49.28          48.41
        NEGATION         34.89          33.83            41.49         37.42          36.77
          1_SHOT         33.29          32.71            40.36         36.26          35.49
```

	dataset	template	concept_type	zero_shot_acc	zero_shot_macro_f1	one_shot_acc	one_shot_macro_f1
0	snli_dev_holdout	base	action	0.368056	0.365210	0.329325	0.165159
1	snli_dev_holdout	base	count	0.333333	0.313066	0.329325	0.165159
2	snli_dev_holdout	base	negation	0.190476	0.126984	0.329325	0.165159
3	snli_dev_holdout	base	object_or_attribute	0.390783	0.346056	0.329325	0.165159
4	snli_dev_holdout	base	spatial	0.327586	0.282306	0.329325	0.165159


Global Zero-Shot Accuracy by Prompt Template
high_specificity
negation_explicit
concept_framing
low_specificity
base
39.97%
39.39%
38.40%
38.38%
37.39%
Prompt Template
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
Mean Zero-Shot Accuracy


Zero-Shot vs. 1-Shot Transfer Learning
Mode
Zero-Shot
1-Shot (Few-Shot)
Accuracy
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
base
low_specificity
high_specificity
concept_framing
negation_explicit
Prompt Template


Template Performance Across Concept Types
Extracted Concept Type
action
count
negation
object_or_attribute
spatial
35.32%
36.59%
36.01%
37.51%
37.73%
28.68%
35.13%
48.40%
38.34%
43.19%
51.44%
39.01%
38.28%
33.83%
33.53%
37.03%
44.24%
34.37%
37.75%
39.21%
34.49%
37.03%
42.77%
44.49%
43.31%
base
concept_framing
high_specificity
low_specificity
negation_explicit
Prompt Template
Accuracy
0.50
0.45
0.40
0.35
0.30

4. Results Analysis & Conclusion

4.1 Overall Trend and Adapter Efficacy

While the lightweight adapter was highly parameter-efficient (~100k trainable parameters), the model's overall zero-shot performance was modest. Across all datasets and templates, the mean zero-shot accuracy was approximately 38.7% with a macro-F1 of 0.27. Since this is a 3-class task, these metrics indicate the model operates only slightly above chance in many settings. This suggests that while the adapter learns some signal from the base prompt during training, it lacks the expressiveness to robustly separate entailment, neutral, and contradiction across new prompt changes without retraining.

4.2 Prompt Engineering Impact

My evaluation proves that prompt wording does alter the model's prediction pathways, though the gains are incremental.

- Top Performers: concept_framing was the best zero-shot template (37.1% accuracy), followed very closely by low_specificity (36.9%) and negation_explicit (36.8%).
- Weakest Performer: The base template consistently performed the worst (34.8%).

This confirms that providing structural visual anchors or asking the model to look for "broad categories" genuinely helps the adapter process the frozen embeddings better than a simple, flat sentence.

4.3 Semantic Mismatch vs. Logical Entailment

The synthetic stress sets revealed a critical insight: the model behaves completely differently depending on the underlying logic.

- Mismatch Neutral (51.5% Acc): The model performed best when the text simply did not align with the image.
- Negation Contradiction (31.0% Acc): The model struggled to detect explicit contradiction.
- Identity Entailment (23.2% Acc): The model completely failed to recognize when the hypothesis was identical to the premise.

These results strongly indicate that the model has learned *coarse mismatch detection* (recognizing when things do not belong together) rather than *fine logical entailment* (recognizing exact semantic equivalence).

4.4 Concept Difficulty and the Negation Bottleneck

Breaking down the diagnostics by concept type exposed severe limitations with polarity reversal. The best-performing categories were count (41.2%), spatial (39.3%), and object_or_attribute (38.2%).

The worst-performing concept by far was negation (29.2%). Even when utilizing the negation_explicit prompt template designed to handle words like "not," the performance remained poor. This proves an architectural bottleneck: because the BERT text encoder is strictly frozen, it cannot adapt its internal linguistic representation to flip the polarity of the sentence, and the lightweight adapter is not expressive enough to reverse the semantic meaning on its own.

4.5 The "Neutral" Fallback (Confusion Bias)

An analysis of the confusion matrix exposes a severe decision boundary bias: when unsure, the model overwhelmingly collapses to predicting neutral. It predicted neutral 41.3% of the time overall. Furthermore, true contradictions were predicted as neutral 43.4% of the time, and true entailments as neutral 36.3% of the time. This is a classic symptom of a weak decision boundary between the three entailment classes.

4.6 One-Shot Instability

Finally, the one-shot adaptation protocol provided only limited gains and proved highly unstable. While the mean one-shot accuracy (39.1%) was slightly higher than zero-shot, the macro-F1 scores often degraded, particularly for the high_specificity and low_specificity templates. Because the support signal was extremely small (only 1-shot per class), the copied adapter frequently overfitted to the few selected examples, damaging its generalization capabilities.

4.7 Conclusion and Architectural Insights

Ultimately, prompt engineering helped marginally, but the main limitation of this pipeline is architectural. By replacing the attention-based cross-modal fusion head from earlier tasks with a bottleneck adapter, I drastically improved training efficiency. However, simply concatenating the ViT and BERT [CLS] embeddings removed explicit image-text interaction learning. This tradeoff successfully preserved broad multimodal compatibility (mismatch detection) but sacrificed the precise relational reasoning required for exact entailment and contradiction.

File Structure

```
Visual_Entailment_Project/
├── task4_pipeline.py              # Contains entire pipeline
├── task_4.ipynb                   # Run the code
├── task4_results.csv              # Aggregated accuracy and F1 metrics for all evaluations.
├── task4_concept_diagnostics.csv     # Row-by-row log of predictions, labels, and errors.
├── prompt_engineering_guide_task4.md # ranking the best templates, concept-specific rules.
├── what_till_we_do_wrong.md  # Failure report detailing worst buckets & frequent confusions.
├── task4_concept_lexicons.json  # Mined dictionary of verbs, nouns, & cues
├── task4_run_summary.json       # Metadata and hyperparameters ensure full reproducibility.
└── task4_swiglu_adapter_best.pth     # Trained weights for Adapter & SwiGLU classifier.
```